

Linux上的异步IO

在 **Linux** 系统里，异步 **I/O** 让程序在等待数据读写时，能继续做其他事情，这样可以提高效率。它主要用三种方法来实现：**select**、**poll** 和 **epoll**。

select、poll 和 epoll

select：这是最早的方式，用来检查多个文件描述符（比如网络连接）哪个准备好进行操作。它有个限制，就是最多只能处理 **1024** 个文件描述符。

poll：比 **select** 更灵活，不受文件描述符数量的限制。但它的效率不高，因为每次都要遍历所有文件描述符。

epoll：这是针对高并发的场景设计的，效率更高。**epoll** 只在文件描述符真的有事件发生时才通知，大大提高了处理效率。特别适合大量连接的情况。

多线程的参与

这些 **I/O** 方法本身不涉及多线程，不过你可以在多线程程序中使用它们。例如，一个线程专门处理 **I/O** 事件，而其他线程处理不同的任务。

自定义的通信协议

自定义通信协议就是你根据需要设计数据交换规则。

消息格式：怎么设计数据包的结构。

数据编码：如何把数据转成字节流。

错误处理：如何应对通信中出现的问题。

流量控制：如何避免数据传输过快造成拥塞。

Linux中一个socket的内存消耗

协议栈：例如，**TCP** 的控制块和缓冲区会占用内存。

文件描述符：每个 **socket** 会占用一个文件描述符。

应用层缓冲区：应用程序可能为 **socket** 设置额外的内存。

通常每个 **socket** 的内存消耗在几百KB到几MB之间，具体情况取决于系统配置和使用的协议。

TCP建立长连接

TCP 建立长连接的过程叫做“三次握手”：

SYN：客户端发出请求，表示想要建立连接。

SYN-ACK：服务器回应，确认收到请求。

ACK：客户端再确认一次，连接建立成功。

建立了连接后，双方就可以开始数据传输了。

Linux的文件系统

目录结构：文件按树状结构组织。

文件系统类型：有很多种，比如 **ext4**、**XFS**、**Btrfs** 等。

权限管理：可以设置文件的读、写、执行权限。

文件系统加载过程

引导加载程序（**Bootloader**）：在系统启动时，**Bootloader**（像 **GRUB**）会加载 **Linux** 内核。

内核启动：内核会初始化硬件，并挂载根文件系统。

文件系统初始化：内核识别并挂载文件系统，开始管理数据。

用户空间初始化：系统启动用户空间的程序，比如 **init** 进程，进一步初始化系统服务和用户环境。

死锁，怎么生成的，举个例子

死锁 是一种程序错误状态，发生在两个或多个进程（或线程）互相等待对方释放资源，结果大家都陷入了无限等待中，程序就卡住了。

生成死锁的条件：

互斥：资源只能由一个进程占用，比如一个打印机一次只能被一个进程使用。

持有并等待：一个进程已经占有某个资源，并且还在等待其他进程释放其他资源。

非抢占：已经占用的资源不能被强制回收，只能等进程自己释放。

循环等待：形成一个进程的环，每个进程等待下一个进程持有的资源。

虚拟地址和物理地址

虚拟地址 是进程使用的地址，物理地址 是实际内存中数据的真实位置。虚拟地址和物理地址之间的关系是通过内存管理单元（**MMU**）和页表来建立的。

如何建立关系：

虚拟地址：程序运行时使用虚拟地址，这些地址是进程认为的地址。

页表：内核维护一个页表，记录虚拟地址和物理地址的映射关系。

MMU：内存管理单元将虚拟地址转换为物理地址，查找页表来找到数据在物理内存中的实际位置。

多线程和多进程的区别

多线程 和 多进程 都是并发执行任务的方式，但它们有一些关键的不同点：

多进程：

每个进程有独立的内存空间。

进程间通信（**IPC**）比较复杂，因为它们不能直接共享数据。

如果一个进程崩溃了，其他进程不会受到影响。

比较适合需要隔离和稳定性的任务。

多线程：

线程在同一个进程中，共享内存空间。

线程间通信比较简单，因为它们可以直接访问共享数据。

一个线程崩溃可能会影响到整个进程。

比较适合需要共享数据和资源的任务

手撕自守数

```
1^1 = 1
2^2 = 4
3^3 = 27
// 函数来检查是否是自守数
bool is_self_power_number(int n) {
    // 计算 n^n
    long long result = 1;
    for (int i = 0; i < n; i++) {
        result *= n;
    }

    // 转换成字符串
    char result_str[100];
```

```
    sprintf(result_str, "%lld", result);

    char n_str[10];
    sprintf(n_str, "%d", n);

    // 获取结果字符串的长度
    int result_len = strlen(result_str);
    int n_len = strlen(n_str);

    // 检查结果的最后 n_len 位是否与 n 相同
    if (result_len >= n_len) {
        return strcmp(result_str + result_len - n_len, n_str) == 0;
    }

    return false;
}
```

Linux上的异步IO了解过吗?

讲一下select/poll/epoll

这中间有多线程的参与吗?

讲一下自定义的通信协议

Linux中一个socket大概有多少内存消耗?

TCP要建立一个长连接，是怎么做的?

Linux的文件系统能描述一下吗

从Linux kernel启动开始，介绍一下文件系统和整个加载的流程

问死锁，怎么生成的，举个例子?

进程的虚拟地址有多少，物理地址和虚拟地址的关系，这两种地址是怎么建立起关系的?

多线程多进程的区别?